FINPILOT

Personal Finance Intelligence

CA3 Mini Project 2 | CSB312 – Generative AI Foundations

Submitted by	Hemant Sihag
Programme	B.Tech Computer Science & Engineering – Generative AI
Institution	Lovely Professional University
Project type	Generative AI-powered expense tracking web application
Deployment	Render
Report date	October 2026

Report covering the original HTML/CSS/JavaScript frontend, FastAPI backend, Gemini API integration, validation, database workflow, and Render deployment.

Abstract

FinPilot – Personal Finance Intelligence is a web application designed to simplify personal expense recording. A user describes an expense in everyday language, for example, “I spent 450 rupees on pizza yesterday using UPI.” The application sends the description to a Python FastAPI backend, which requests structured extraction from the Google Gemini API. The extracted details are validated and shown as a draft for the user to review. A transaction is saved only after the user confirms it. The application also provides a transaction ledger, update and delete operations, and monthly category summaries.

1. Introduction

Recording daily expenses can be time-consuming when every transaction must be entered into separate fields. FinPilot uses Generative AI to convert a natural-language description into structured transaction fields. The user remains in control through a review-and-confirm step before the record is stored.

2. Objectives

• Allow users to describe expenses in simple natural language.

• Extract amount, currency, category, merchant, date, and payment method.

• Validate the AI-generated draft and allow the user to review it before saving.

• Store and display transaction records in a ledger.

• Support creating, reading, updating, and deleting transactions.

• Provide monthly spending totals grouped by category.

• Deploy the application as a web service.

3. Technology Stack

Technology	Purpose
HTML, CSS, JavaScript	Frontend structure, styling, interactions, and HTTP requests.
Python + FastAPI	Backend application and REST API endpoints.
Google Gemini API	Extracts transaction information from natural-language input.
HTTPX	Sends HTTP requests from the backend to the Gemini API.
Pydantic	Defines data schemas and validates transaction fields.
SQLAlchemy	Provides database access and ORM operations.
SQLite / PostgreSQL	Stores transactions; SQLite for local development and PostgreSQL when configured.
Render + GitHub	Hosts the service and stores the source code repository.

4. System Architecture

The frontend communicates with the FastAPI backend using HTTP requests. For extraction, the backend calls the Gemini API, parses the returned structured response, and validates it. The extracted result is returned as a draft to the frontend. After the user confirms, the backend validates the submitted transaction and uses SQLAlchemy to persist it.

Layer	Responsibility
Frontend — static/index.html	Expense input, draft preview, user edits, confirmation, ledger, and summaries.
Backend — app/main.py	Routes requests to extraction, validation, transaction, and summary logic.
AI service — app/ai.py	Builds the prompt and calls Gemini using HTTPX; processes the response.
Schemas — app/schemas.py	Defines expected fields and validation rules for transaction drafts.
Database — app/db.py	Configures database access and SQLAlchemy operations.

5. Application Workflow

• **Step 1 — Input:** The user enters a natural-language expense description.

• **Step 2 — Request:** JavaScript sends the description to `POST /api/extract`.

• **Step 3 — AI extraction:** The backend sends a prompt to Gemini and requests structured transaction data.

• **Step 4 — Validation:** The response is parsed and checked against the expected schema.

• **Step 5 — Preview:** The frontend displays a draft for the user to review and edit.

• **Step 6 — Confirmation:** The user confirms the details; the backend validates the transaction again.

• **Step 7 — Persistence:** The backend saves the confirmed record to the configured database.

• **Step 8 — Management:** The user can view the ledger, update or delete transactions, and review monthly summaries.

6. Data Fields and Validation

The extraction flow is designed around fields including **is_transaction**, **amount**, **currency**, **category**, **merchant**, **date**, **payment_method**, **confidence**, and **missing_fields**. Supported categories include Food, Travel, Shopping, Bills, Health, Entertainment, and Other. Payment methods include Cash, UPI, Card, and Other. Validation checks the expected format but cannot guarantee every AI inference is factually correct.

7. API Endpoints

Method and endpoint	Purpose
GET /api/health	Check application health.
GET /api/config	Return frontend configuration.
POST /api/extract	Extract a transaction draft using AI; does not save it.
POST /api/validate	Validate a draft edited by the user.
POST /api/transactions	Create a confirmed transaction.
GET /api/transactions	Retrieve the transaction ledger.
PUT /api/transactions/{txn_id}	Update an existing transaction.
DELETE /api/transactions/{txn_id}	Delete a transaction.
GET /api/summary	Return monthly totals grouped by category.

8. Database and Transaction Management

SQLAlchemy is used to interact with the database. SQLite supports local development without a separate database server. When a PostgreSQL connection is supplied through the DATABASE_URL environment variable, the application can use PostgreSQL in the hosted environment. The transaction API supports CRUD operations: Create, Read, Update, and Delete.

9. Deployment

The source code is maintained in GitHub and the web service is deployed on Render. The deployment configuration installs Python dependencies from `requirements.txt` and starts FastAPI using Uvicorn. Environment variables include the Gemini API key, model name, and optional database connection URL. Secret API keys should never be committed to the repository.

Live application: https://finpilot-nri3.onrender.com

Source repository: https://github.com/hemantsihag007/gen_ai_ca3.git

10. Testing Plan and Verification Status

The following checks should be performed on the deployed application before submission. This report does not claim that every live test has passed; record results after testing the current deployment.

Test	Expected result	Status
Open application	Page loads successfully.	To verify
Extract a valid expense	AI returns a draft with expected fields.	To verify
Review and save	Confirmed transaction appears in ledger.	To verify
Edit transaction	Updated values appear in ledger.	To verify
Delete transaction	Selected record is removed.	To verify
Monthly summary	Category totals reflect saved transactions.	To verify
Persistence	Records persist if a persistent database is configured.	To verify

11. Limitations

• AI extraction may misinterpret incomplete, ambiguous, or unusual expense descriptions.

• The user review step reduces risk but does not eliminate incorrect data.

• Gemini API availability, quotas, model access, network conditions, and API key configuration can affect extraction.

• Database persistence depends on the configured database; local SQLite should not be assumed to persist across all hosting resets.

• The application is an expense-recording aid, not a financial adviser or bank-connected financial service.

12. Future Enhancements

• Add receipt image upload and OCR-based extraction.

• Provide monthly budgets and spending alerts.

• Add charts for spending trends.

• Improve authentication and user-specific transaction isolation.

• Add automated tests and monitoring for API/database failures.

• Support CSV export and downloadable monthly reports.

13. Conclusion

FinPilot demonstrates how Generative AI can be integrated into a practical web application. It converts natural-language expense descriptions into structured drafts, validates the data, and lets users confirm transactions before saving. The project combines frontend development, FastAPI, external API integration, schema validation, database operations, and cloud deployment. Further testing can improve reliability, persistence, and usability.

14. Viva Questions

What is FinPilot?

A Generative AI-based expense tracker that converts natural-language descriptions into structured transaction drafts.

Why use Gemini?

To extract transaction fields from text using a Generative AI model.

What does FastAPI do?

It provides backend routes for extraction, validation, transaction management, and summaries.

Why require confirmation?

AI can make mistakes, so the user reviews the draft before it is saved.

What is CRUD?

Create, Read, Update, and Delete operations on transactions.

What does Pydantic do?

It validates that data follows expected schemas and field rules.

How is the app deployed?

The source is in GitHub and the FastAPI service is deployed on Render.